MLP Unchained

January 12, 2026

Mario Prignano Gabriele Cirillo Alessio Morosini Lorenzo Lausi

Eterna

Abstract

Attention has revolutionized modern Deep Learning through effective token mixing (Vaswani et al., 2017). However, this power comes at a cost: Self Attention scales quadratically with sequence length. Consequently, recent years have seen multiple attempts to replace attention entirely with more efficient methods.

The Deep Learning community has focused primarily on improving spatial mixing efficiency, while the MLP FFN following Attention in Transformer architectures has received comparatively less attention.

In this work, we adopt a unified view of Key-Value memories for both Self-Attention and MLPs, conceptualizing the transformer as a fully attentional network where input first attends to itself (self-attention), then to a learned memory bank (MLP).

We show that MLPs play a crucial role in modern Seq2Seq architectures by showing that spatial mixing (self-attention) can be heavily approximated or nearly removed, while MLPs (learnable-memory interrogation) are difficult to approximate. We also propose a simple, fast, easily implementable mixing technique based on low-rank mixing, applicable to both causal and non-causal spatial and channel mixing. Using also this novel mixer, we systematically replace first Self-Attention, then MLPs, revealing how much predictive work MLPs actually perform. Finally, we introduce a mathematical framework to rigorously support our findings. Code for our project is available at: Code in Vathos Repository

Kaggle Notebook

Email: Mario Prignano < prignano.2122585@studenti.uniroma1.it>.

Machine Learning 2025, Sapienza University of Rome, 2nd semester a.y. 2024/2025.

1. Mixing and Attention Recap

Sequence modeling has acquired a central role in modern Deep Learning, since most of the data types (Text, Music, DNA, Images...), can be interpreted in the form of sequences. Attention has easily conquered this big field, being central in the famous Transformer Architecture (Vaswani et al., 2017), which from Vision Transformers (Dosovitskiy et al., 2020), to Diffusion Transformers (Peebles & Xie, 2023), or Transformer based LLMs dominates the Deep Learning research.

Seq2Seq models Given the embedded sequence of vectors $x_t \in \mathbb{R}^d$, we can construct an input matrix $X \in \mathbb{R}^{L\times d}$ where L is the sequence length, and d is what in the deep learning field is called *Embedding Dimension*. In general, a sequence model is defined by a parametric function F_θ where θ are the model parameters, which links the input sequence X to an output sequence $O \in \mathbb{R}^{L\times d}$ via:

$$O = F_\theta(X)$$

Usually F is the composition of multiple layer functions $f^{(l)}$ to iteratively modify the input sequence, where $l = 1, ..., n_layers$ is the number of the layer. Since for most of the modern models, the structure of $f^{(l)}$ remains unchanged -so that each layer is only different in its parameters- we are going to consider a single layer l, whose input X will be the output of the layer before. Given that X is composed of L d-dimensional vectors x_t ($t = 1, ..., L$), it is natural to see a layer output $Y \in \mathbb{R}^{L\times d}$ as L d-dimensional vectors y_t. In the case that each y_t depends only on previous x_i vectors, with $i = 1, ..., t$ we say the layer is **causal**.

Given the input vectors, two types of f are usually combined in every model: *Spatial mixing*, and *Channel Mixing*.

Linear Spatial Mixing All Linear Spatial Mixers can be seen as setting

$$Y = f(X)_\theta = WX$$

where $Y \in \mathbb{R}^{L\times d}$ is the output sequence, and $W \in \mathbb{R}^{L\times L}$ is the Spatial Mixing Matrix. Transformers, SSMs like

Mamba (Gu & Dao, 2023) or s4 (Gu et al., 2022), Hyena (Poli et al., 2023), Linear Transformers (Katharopoulos et al., 2020), MLP-Mixer (Tolstikhin et al., 2021), Retention (Sun et al., 2023), Monarch Mixers (Fu et al., 2023)... all of these models are usually defining a different spatial mixing matrix W, for example Toeplitz Matrix in the case of long convolutions, Attention in the case of transformers. Naively the cost of applying an L by L matrix W to X is $O(dL^2)$, but there exist classes of matrices whose matmul with X has an efficient alternative algorithm that opens the possibilities to subquadratic spatial mixers.

Causal masking To make every y_t depend only on x_i vectors, with $i = 1, ..., t$, the W in $Y = WX$ must be a lower triangular matrix, often in deep learning models this is achieved by simply masking W, formally: $W_{masked} = W \odot G$ where $G_{ij} = 1 \;\; if \; i \geq j \;\; else \; 0$ and $\odot$ is the Hadamard pointwise product.

Self Attention, memory on sequence length Self Attention is a particularly effective choice for a spatial mixer (Vaswani et al., 2017), where we set (we show a simplified version)

$$Y = softmax(QK^T)V$$

in the causal settings, or other cases:

$$Y = softmax(QK^T \odot G)V$$

and we can say that the mixing matrix is

$$W = softmax(QK^T \odot G)$$

where, $Q \in \mathbb{R}^{L \times d}$, and $K, V \in \mathbb{R}^{M \times d}, W, G \in \mathbb{R}^{M \times L}$. In the case Attention is used as a spatial mixer, namely Self Attention, Q, K, V are all derived from the input X, usually with a simple linear layer, and $M = L$, $Q = XW_q, K = XW_k, V = XW_v$ where $W_q, W_k, W_v \in \mathbb{R}^{d \times d}$. It is also always useful to see Q, K, V as a collection of $d-$dimensional vectors q_t, k_t, v_t, with $t = 1...L$

The power of Attention, usually is explained via associative Key-Value KV memory. Essentially, Q can be seen as a set of queries, interrogating a memory V, indexed by K. The non linearity, usually $softmax(\cdot)$ serves to make the query key interaction $q_i k_j^T$ less noisy. In fact if $L, (M) >> d$ there will be the possibility of having L different queries, attending to $L, (M)$ different keys, via d-dimensional dot products; and since there can be at most d different linearly independent k_j given a q_t the querying will degrade into a noisy interaction. Here $softmax$, or other non linearities, can help by introducing an heuristic on how to choose which interaction to activate more, for example the softmax heuristic is that the strongest $q_i k_j^T$ are not part of the noise, while most of the interactions are just noise.

The output, of course, is not a single vector v_i, but a linear combination of all the row vectors v_t of V, where as we said weights are the similarity of rows of Q and K activated by the non linear function. In the case of Self Attention this interpretation becomes an interrogation of the sequence by the sequence itself, and can be actually seen as a message passing over the sequence length.

The G matrix, which usually serves only as causal masking, is also interesting from a positional encoding and bias perspective. Retentive networks (Sun et al., 2023), and modern Linear Attentions (Katharopoulos et al., 2020) usually learn G or derive it from the input to enhance the expressivity of the Attention mechanism.

Channel Mixers, MLP as a key-value memory The other type of mixing, Channel Mixing, is usually handled by an unbiased MLP:

$$Y = f_\theta(X) = \sigma(XW_1)W_2$$

where $W_1, W_2 \in \mathbb{R}^{M \times d}$ and σ is a pointwise non linearity (including softmax), and M is often called $hidden \;\; dimension$. Often the utility of MLP is explained by its creation of very informative and dense latent representation. Here we adopt another explanation, using Attention itself. Indeed if we rename $K = W_1$ and $V = W_2, Q = X$, the unbiased two layer mlp formula becomes:

$$Y = \sigma(QK^T)V$$

So our easiest explanation of what MLP is doing in modern sequence modeling architecture, is a learned Key-Value memory bank, with M slots, queried by the input itself X. Furthermore modern transformers, like Llama (Touvron et al., 2023) usually adopt SwiGLU (Shazeer, 2020) as the activation function σ, which is very similar to learning data dependent gate G, finding again, the full attention formula:

$$Y = \sigma(QK^T \odot G)V$$

One could argue that a deeper MLP with more than 2 layers is not explainable like this. This theory in fact predicts that a deeper MLP has no direct meaning. It can be thought of as a chained memory, but a chained memory is harder to both interpret and train with standard optimization techniques, and hence is not beneficial. Another prediction this theory makes, is that in many cases, increasing M is beneficial, since its making the memory bigger, without increasing gradient complexity. Here's what we found by the later explained experiments with Transformers;

Table 1. Results on MAESTRO (Hawthorne et al., 2018), sequence length 2116, 90 epochs

Model	Perplexity
(16mln) MLP depth: 2, $M = 2$	7.17
(17mln) MLP depth: 2, $M = 3$	7.03
(24mln) MLP depth: 3, $M = 2$	7.31

One could argue that to increase depth while retaining the same training complexity, its just enough to insert skip connections. In fact this theory also predicts it, since for example:

$$Z = X + \sigma(Q_1 K_1^T) V_1$$
$$Y = \sigma(Z K_2^T) V_2$$

can be seen as simply interrogating two Key Value memories sequentially (first adding to the input the first result, and then interrogating a memory again, adding the second result). Hence the maximum non-skip depth predicted is 2, and it perfectly fits the state of the art choices for the general Transformer architecture.

2. Kronecker Operator

As we said before, one can be tempted to approximate both spatial and channel mixer with low rank matrices. This, though, introduces several problems, as for example, the impossibility of creating a causal low rank mixer by setting $W = uv^T$. Here we progressively show how we derived our Kronecker Operator, which solves these problems effectively, with low computational costs, and also helps us better understand modern Linear Attentions and SSMs.

Kronecker Product The Kronecker product is a particular tensor product defined as follows: Given $A \in \mathbb{R}^{n \times m}$ and $B \in \mathbb{R}^{p \times q}$ matrices

$$A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{pmatrix} \in \mathbb{R}^{np \times mq}$$

Rank 1 This product has some very useful properties, for example the rank of $A \otimes B$ is the product of the rank of A and the rank of B, and if the rank of A and B is full, then the rank of $A \otimes B$ is full. Nevertheless the Kronecker product can be seen as a reshaped Rank 1 matrix.

The product is also very predictable: the structure of the second matrix reflects on each block, meanwhile the structure of the first matrix scales each block differently. An example of the Full Kronecker product can be seen in 1

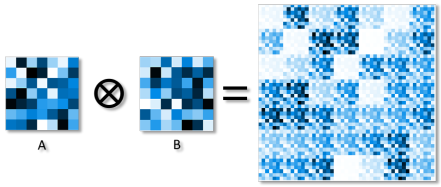

Figure 1. Full Kronecker Product

Note: More details on the precise derivation of the Kronecker Operator as a causal Mixer are in the Appendix A, as well as the implementation of the subquadratic algorithm to compute $(A \otimes B)X$

2.1. Connection with Linear Attention

The kronecker operator we just defined, can be seen as applying a learned approximation of a 1d convolution, which is the general case of a 1d cumulative sum over the sequence length. In fact, we observe that the trained mixing matrices tend to approximate a Toeplitz structure with a decaying kernel. This induces an implicit heuristic on temporal dependency, whereby each x_t receives information —via simple additive accumulation— primarily from its nearest neighbors in time, with progressively diminishing contributions from more distant positions. An idea that was also used by the retention mechanism. Convolutions also have been playing a central role in the developing of faster subquadratic attention substitute, for example with Hyena and S4. Finally we suppose in future our mixer can be seen as a simple way to make the cumulative sum operation GPU-Friendly with ease, in the case of linear attentions:

In causal linear attention, the output at time t is defined as

$$y_t = \frac{q_t^\top \sum_{i=1}^{t} k_i v_i^\top}{q_t^\top \sum_{i=1}^{t} k_i}.$$

Defining the state matrix

$$S_t = \sum_{i=1}^{t} k_i v_i^\top,$$

it follows immediately that the computation admits the recurrence

$$S_t = S_{t-1} + k_t v_t^\top,$$

with $S_0 = 0$. Therefore, linear attention reduces to a simple cumulative sum.

2.2. Expressivity

In our work, we often stumbled upon the need of measuring the representative power of a constrained set of matrices, similarly to what they did in Monarch (Dao et al., 2022a). Hence, when we approximated MLP and Attention Linears with our mixer it came naturally to look for a better measure of performance degradation. So we introduced a small mathematical framework to do so, defining a simple measure of expressivity: Given a particular set of matrices S, an expressivity function E should be like:

$$E(S) : S \subseteq \mathbb{R}^{N\times N} \to [0,1] \subset \mathbb{R}$$

Our expressivity function (with P being the projection that maps every $W \in \mathbb{R}^{N\times N}$ on S):

$$E(S) = 1 - \mathbb{E}_W[\|W - P_S(W)\|]$$

This expressivity function is easy to compute, thanks to the Central Limit Theorem, and it has useful additional properties, which we better explain in the Appendix C, where we show also some empirical evaluation of E on S being the Monarch Matrices or S being the Matrix obtainable by Kronecker product.

3. Results

In our experiments, we both tried to replace the spatial mixer, and channel mixer in the Transformer Architecture with the Kronecker Operator and Kronecker product, obtaining very different results.

We trained our models with AdamW, for around 90 real epochs, on the Maestro Dataset, with sequence length 2116. on 2 T4 GPUs. We want to make clear that most of the experiments we did, were meant for another scope, and what we find is highly data dependent. We suppose that training on way bigger Natural Language Dataset, where static memory is far more important, could enhance the differences.

3.1. Replacing Channel Mixer

When we tried to set the W_1 and W_2 of a classical MLP in a transformer with a simple kronecker product.

$$Y_{channel} = \sigma(XW_1)W_2$$

where $W_1 \in \mathbb{R}^{d_1\times d_2}$ and $W_2 \in \mathbb{R}^{d_2\times d_1}$ We tried to replace the Linear Maps W_1, W_2 both with a Kronecker Product. With this setting, model performance, dropped heavily. We also may want to note that the we suppose Linear Maps in attention are interfering with results, as they effectively absorb part MLP work (investigable in future). Explaining this result can be done via the Key-Value interpretation, indeed as previously said, the actual rank of a Kronecker product of this type is 1 (by a tensor reshape), although gradient is far more complex, so we included also a simple low rank MLP in the results table.

So if we think of W_1, W_2 as K, V, we are actually constraining our memory V to be highly repetitive. Hence, the layer can effectively memorize almost a single vector.

3.2. Replacing Spatial Mixer

We were surprised to see, what happened when instead we replaced the spatial mixing matrix (Self Attention) W with the Kronecker Operator, both in causal and non-causal settings. The results were not degraded as much as they were with replacing the channel mixer. Also note that other Low Rank, and simplified attention are showing the same results, for example Linear Attention. Other than the numerical proofs we used a Kronecker Operator based Model to generate Jazz (PiJAMA) and Classical (MAESTRO) MIDI samples (Hawthorne et al., 2018), with fairly discrete results (against our expectation): Drive Folder of Generations.

In practice we kept the channel mixer as an MLP, and replaced the attention in the transformer with a short convolution gated Kronecker Operator.

Table 2. Results on MAESTRO (Hawthorne et al., 2018), sequence length 2116, 90 epochs

Model	Perplexity
(16mln) Full Transformer	7.17
(16mln) Linear Transformer	8.12
(10mln) Kronecker Spatial mixers	8.31
(10mln) Low Rank Channel mixers	9.77
(10mln) Kronecker Channel mixers	10.23

In future we may want to analyze also how much can this memory framework expanded, how to measure memory, as well as how fast do this memory fill. In future we might also want to better analyze the strict connection between our mixing, SSMs, and Linear Attention, since our mixer can be seen as an efficient learned cumulative sum approximation, a deduction that is supported by observing the trained matrices.

For now we developed a basic and initial yet rigorous mathematical framework to explain these differences, in the appendix.

Machine not learning To push the idea that fine grained spatial mixing is less important than detailed channel mixing even further, we tried initializing some of these matrices, or even all of them, to famous repetitive matrices (like

Toeplitz) and even totally fixing them, freezing the weights.

Some of the results (shown in the appendix B) remark the idea that learning highly expressive spatial mixing matrices might not be this important when compared with learning unconstrained channel mixing matrices. Thus the non linear channel mixer can do the hard work quite well, when provided with a simply mixed signal. Additional informations about these findings are in Appendix B and C

Is it a matter of distributions? We tried explain what makes the performance drop so quickly when we try to train the model with approximated MLPs using Low Rank solutions, and why is it so easier to replace the spatial mixing leveraging these solutions. So we analyzed the distribution of the values of linear layers matrices in the MLP, in Attention and in our Mixer. Then we looked to the singular values distributions of these matrices. The range of values assumed by the matrices of trained Kronecker Operators tends to be mostly positive. What emerges, and better shown in Appendix D, is that while attention is structurally low rank, and positive (since it is a stochastic matrix), easily approximable with Kronecker, the MLPs of a Transformer, naturally tends to be full rank with values distributed with Gaussians distributions; a result that fits very well with the static Key-Value memory interpretation: If the MLP needs to store literal facts from the dataset, it requires to fit all of these specific information in vectors of the memory bank V (to recall $V \in \mathbb{R}^{M \times d}$), and to efficiently extract facts from a static memory it comes natural to separate the vector in the space as much as possible, leading to very high ranks. Here's an intuitive geometric idea on why in Self Attention the rank of the Query-Key correlation matrix $softmax(QK^T)$ is pretty low.

Attention is a stochastic matrix, and we can assume that it is derived from randomly distributed inputs so we can exclude from this discussion particularly bad cases. Self Attention matrix is also L by L, and so its size increases with sequence length. Every entry of this matrix is strictly bounded in $(0, 1)$, hence the space in which these matrices live is $(0, 1)^{L \times L}$. An intuitive measure of rank is the average angle between vectors. And it is intuitive that the expected angle between random vectors taken in this space is low. A simple example of this, comes when we look for two orthogonal vector in $(0, 1)^2$ where the only possible choice is the canonical base e_1, e_2 , and hence the probability of receiving two orthogonal vectors in the input are 0, and when L increases the effect is strongly enhanced and angles naturally decrease.

In the Appendix C we show empirical results of this.

3.3. On Vision things are even better

For the non causal setting we choose to replace the Self Attention in a Tiny ViT, with our non causal Kronecker Operator. In this case we removed both local 1d convolution and gate. As baseline here we used a Vision Transformer equipped with FlashAttention (Dao et al., 2022b). Since we trained on CIFAR10 (Krizhevsky et al., 2009) and CIFAR100 the sequence length L was only 65, and we could not fully benefit from our subquadratic time complexity but still be comparable with FlashAttention speed. The architecture here is the same for both the models, the only change was in the Spatial Mixing, specifically:

- CIFAR10: 5 layers, 128 embedding dimension,
- CIFAR100: 5 layers, 256 embedding dimension

Table 3. Results on CIFAR10, sequence length 65, 100 epochs

Model (params)	**Accuracy %**	**time**
FlashAttention (Dao et al., 2022b) (1)	81	13s
Kronecker (0.69)	78	13s
Rank 1 Mixer (0.69)	77	12s
Rank 1 Channel Mixer (1)	56	12s

Table 4. Results on CIFAR100, sequence length 65, 100 epochs

Model (params)	**Accuracy %**	**time**
FlashAttention (Dao et al., 2022b) (4)	56	22s
Kronecker (2.5)	54	18s
Rank 1 Spatial Mixer (0.69)	52	12s
Rank 1 Channel Mixer (0.69)	37	20s

On why on vision tasks our spatial mixer performs better, we did not explore further. We suppose it's only fault of the lower length of the sequences in experiments, and its closeness with MLP-Mixer Architecture.

4. Conclusion

In this work we highlight differences in modern Spatial Mixing and Channel mixing, seeing them from the key-value memory point of view, effectively showing how modern state of the art seq2seq architectures, can be seen as intelligently aggregated memories. Finally, thanks to the results we obtained using also our novel Kronecker Operator, we tried giving an intuitive, and then rigorous mathematical explanation on why the channel mixer should remain untouched, while Attention approximations are still to be studied, and are key to newer hybrid architectures.

References

Dao, T., Chen, B., Sohoni, N., Desai, A., Poli, M., Grogan, J., Liu, A., Rao, A., Rudra, A., and Ré, C. Monarch: Expressive structured matrices for efficient and accurate training, 2022a. URL https://arxiv.org/abs/2204.00595.

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pp. 16344–16359, 2022b.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020.

Fu, D. Y., Epstein, S., Nguyen, E., Thomas, A. W., Zhang, M., Dao, T., Rudra, A., and Ré, C. Monarch mixer: A simple sub-quadratic gemm-based architecture. *arXiv preprint arXiv:2310.12109*, 2023.

Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

Hawthorne, C., Stasyuk, A., Roberts, A., Simon, I., Huang, C.-Z. A., Dieleman, S., Elsen, E., Engel, J., and Eck, D. Enabling factorized piano music modeling and generation with the maestro dataset. In *International Conference on Learning Representations*, 2018.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pp. 5156–5165. PMLR, 2020.

Krizhevsky, A., Hinton, G., et al. Learning multiple layers of features from tiny images. *Toronto, ON, Canada*, 2009.

Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4195–4205, 2023.

Poli, M., Massaroli, S., Nguyen, E., Fu, D. Y., Dao, T., Baccus, S., Bengio, Y., Ermon, S., and Ré, C. Hyena hierarchy: Towards larger convolutional language models. In *International Conference on Machine Learning*, pp. 28043–28078. PMLR, 2023.

Shazeer, N. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

Tolstikhin, I. O., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., Yung, J., Steiner, A., Keysers, D., Uszkoreit, J., et al. Mlp-mixer: An all-mlp architecture for vision. In *Advances in Neural Information Processing Systems*, volume 34, pp. 24261–24272, 2021.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in neural information processing systems*, volume 30, 2017.

5. Appendix

A, Details on Kronecker Operator Derivation

Subquadratic Algorithm Given $A, B \in \mathbb{R}^{\sqrt{L}\times\sqrt{L}}$, $X \in \mathbb{R}^{L\times d}$, to compute the product $Y = (A \otimes B)X$ naively one can materialize $A \otimes B$ and apply it to X with a time complexity of $O(L^2)$.

However, we adopt a more efficient way that lowers the cost to $O(2L\sqrt{L})$. Indeed we use the equivalence:

$$(A \otimes B)X = (A\mathbb{X}B^T).\text{reshape}(L, d), \tag{1}$$

$$\text{where} \quad \mathbb{X} = X.\text{reshape}(\sqrt{L}, \sqrt{L}, d) \tag{2}$$

The computational cost of $A\mathbb{X}B^T$ is the sum of the costs of:

$$(1)\ \ Z = \mathbb{X}B^T \quad O(L\sqrt{L}), \qquad (2)\ \ Y = AZ \quad O(L\sqrt{L})$$

So the total cost is $O(2L\sqrt{L})$.

In PyTorch this can be implemented as:

```
Xm = X.view(Bbatch, n, n, d)
Y = torch.einsum('ij,bjkf,lk->bilf', A,
    Xm, B)
Y = Y.reshape(Bbatch, L, d)
```

In the actual code $X \in \mathbb{R}^{Bbatch,L,d}$, but the batched version is equivalent.

Causal Kronecker Mixers To use this algorithm we could simply set the W in $Y = WX$ to be the order 2 Kronecker product $A \otimes B$ with the generic $A, B \in \mathbb{R}^{\sqrt{L} \times \sqrt{L}}$, where if we want we can make A, B learnable matrices.

But a big problem arises when causality is needed, since we need the Kronecker product result to be lower triangular. To show why this is a problem we prepared a simple example.

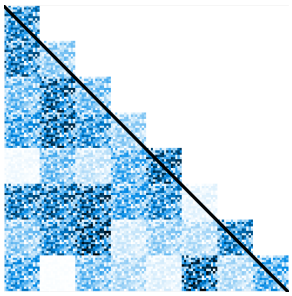

Figure 2. $A \otimes B$ with A tril and B dense

This parametrization is trivially not enough, since its not really lower triangular, and to overcome this problem, a simple solution is to set both A and B to lower triangular matrices: 3

Next we show how we derived the generic order 2 Kronecker Operator parametrization with $A, B \in \mathbb{R}^{\sqrt{L} \times \sqrt{L}}$. (All of the bigger orders are simple reiteration of this concept)

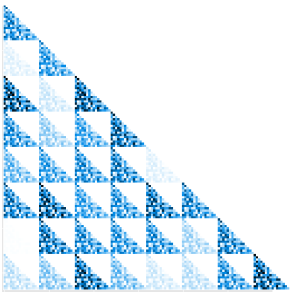

Figure 3. $A \otimes B$ with A tril and B tril

To enhance this parametrization, and build a fully lower triangular W, we choose to make a sum of kronecker products, formally:

$$W = A \otimes B + A' \otimes B'$$

where A is lower triangular, B is lower triangular, A' is lower triangular without the diagonal, and B' is dense, 4

To further augment the expressivity, and maintain low computational costs, we initially decided to adopt diagonal

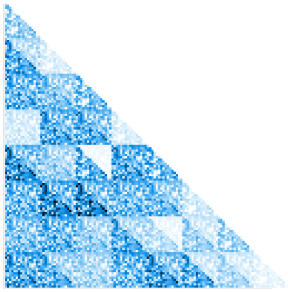

Figure 4. $A \otimes B + A' \otimes B'$

scaling:

$$W = D_1(A \otimes B)D_2$$

where $D_1, D_2 \in \mathbb{R}^{L \times L}$ with $D_1 = diag(d_1)$, $D_2 = diag(d_2)$ and whose product with X, DX can be applied in $O(Ld)$ with pointwise product $X \odot d$, but after the experiments we decided to only use a pointwise product with $\tilde{d}_2 \in \mathbb{R}^{L \times d}$. To make $\tilde{d}_2 \in \mathbb{R}^{L \times d}$ powerful, we did not parametrize it, instead we derive it from a linear layer, and we apply an optional pointwise non-linearity $\sigma(\cdot)$

$$\tilde{d}_2 = \sigma(XW_d), \;\; W_d \in \mathbb{R}^{d \times d}$$

. To this point, the Kronecker Operator looks like this:

$$WX = (A \otimes B)(X \odot \sigma(\tilde{d}_2))$$

Low Rank Causal Mixing We want to make clear again that this parametrization is as expressive as a Rank 2 Causal mixer, which cannot be built by the simpler $W_r = UV^T$ with $U, V \in \mathbb{R}^{L \times 2}$. That is because every lower triangular matrix, whose diagonal elements are non zeros, has full rank, so there is no way to build a lower triangular, rank 2 matrix $W_r = UV^T$. More results about the expressivity and ranks of mixers are in Appendix C.

Non Causal Kronecker Mixers In non-causal settings, meanwhile, we do not need to parametrize W as tril, so we are free of making all of the kronecker matrices dense. This non-causal mixing can be applied to both spatial mixing (like vision tasks), and general channel mixing.

B. Focus only on a few things at a time

Fixed our model to an order 2 causal Kronecker Operator, we observed some pretty interesting behaviours when initializing, or even fixing matrices to particular structured matrices, from simply diagonal, or lower triangular matrices, to causal Toeplitz matrices. Given $X \in \mathbb{R}^{L \times L}$ and having the $maxlen$ fixed to L_{max} (multiple of 2), we define $n = \sqrt{L_{max}}$. So in an order-2 Kronecker Operator we need at most 4 matrices $A, B, A', B' \in \mathbb{R}^{n \times n}$ These matrices are usually initialized with common initialization

techniques and trained. Here we explore what happens if we change the initializations, completely freeze them, or even using only 2 of the 4 matrices.

Order-2 Kronecker Operator To recall our typical o2 parametrization is:

$$Y = WX = (A \otimes B + A' \otimes B')X$$

To obtain a dense, lower triangular mixing matrix, our standard parametrizations is:

$$W = \underset{\text{tril(diag=-1)}}{A} \otimes \underset{\text{dense}}{B} + \underset{\text{diag}}{A'} \otimes \underset{\text{tril}}{B'}$$

Hierarchical Kronecker The cost of the Standard o2 parametrization, is asymptotical to $O(L\sqrt{L})$ but changes with the number of kronecker product needed each time, and goes down whenever we parametrize some matrices as diagonals. However we observed that by simply using:

$$W = \underset{\text{tril}}{A} \otimes \underset{\text{tril}}{B}$$

parametrizing only A and B

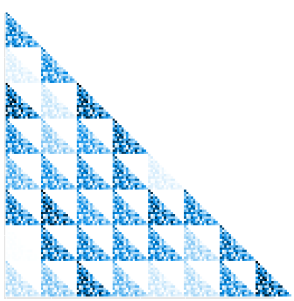

Figure 5. $A \otimes B$ with A tril and B tril

This matrix is a block lower triangular with lower triangular blocks (LTLTB) with each block of side $\sqrt{L}$ and Even if the resulting W matrix is less expressive, and quite sparse, the speed is increased, and the model performance are comparable to the standard one: Loss is the same (on PiJAMA and MAESTRO) and generations are better. Training those matrices, we have a W per layer, and we can see what they are converging to every time: first layers usually tend to have denser As and Bs, while final layers tend to be almost diagonal. In the mid layers, we observe the tendency to have an A pretty similar to a diagonal dominant Toeplitz matrix. We suppose that what is making this structure powerful, is **Structured Sparsity**, and **Hierarchical mixing**. The sparsity comes from the big number of 0 triangles that appears, and its responsible for a less blurry mixing since every y_t depends only from a few x_t, while the LTLTB structure can be seen as applying smaller causal mixing to each $\sqrt{L}$ subsequence, and then summing them together, and all of the operation can, indeed, be seen as a mixture of local and global mixing. To prove that the structure is important we tried experimenting with training A and B to be randomly masked (changing a fixed mask, per layer), with a number of 0s to an half of the total, which is the number of 0s of the LTLTB structure. Then since randomly masking B can put 0s to the diagonal of B (which directly is placing 0s on the diagonal of W), we forced the diagonal of B to be unmasked. Results are reported in 5

Toeplitz Toeplitz matrices represent convolutions. A lower triangular Toeplitz matrix is a causal 1d convolution. Given a convolutional kernel k of length L we say $T_c(k)$ is the causal Toeplitz matrix derived from k. With $L = 3$ for example:

$$T(k) = \begin{pmatrix} k_1 & 0 & 0 \\ k_2 & k_1 & 0 \\ k_3 & k_2 & k_1 \end{pmatrix}$$

In our experiments we usually generate the kernel as $k_t = (t+1)^q$ where q is a small negative number like -0.5

Another direction one could parametrize Toeplitz matrices, is by learning the kernel, which also is pretty low in parameters. If the matrix becomes large enough one could use the FFT to accelerate the computation, but it has several GPU-side optimization problems, and since the side of each $A_i = T(k_i)$ is $\sqrt[2]{L}$ the practical numbers are usually pretty small.

C. Expressivity

To start, we needed to give a rigorous definition for expressivity of a class of matrices.

In the deep learning context indeed we often look for a set of matrices whose matmul is efficient, but we would also like to preserve the learning power of fully dense matrices.

But how can we measure the *learning power* of a particular class of matrices?

Some methods already exist, but we come up with a novel solution, and the following discussion will lead you to its derivation.

Given a set of matrices $S \subseteq \mathbb{R}^{N \times N}$, let $W \in \mathbb{R}^{N \times N}$ be an arbitrary matrix, $P_S(W) := \underset{A \in S}{argmin}(\|W - A\|)$ is the projection of W on S.

Consider:

$$\mathbb{E}[\|W - P_S(W)\|]$$

which is measuring how bad a class of matrices S can approximate every possible matrix of $\mathbb{R}^{N \times N}$. This measure has some problems, for example it is not bounded and so

Table 5. Full vs Randomly Sparse vs Totally Sparse

TYPE	Val Loss	Train Loss	Gen Quality (0–5)
Standard	2.19	2.01	3
Randomly 0.5 Sparse (diagonal can have 0s)	2.24	2.03	2
Randomly 0.5 Sparse (diagonal is always present)	2.21	2.01	3
LTLTB	2.19	2.02	5

it is difficult to interpret. To solve this issue we choose to consider (without losing generality) not the full $\mathbb{R}^{N\times N}$ space, but only the surface of the unit sphere in L_2 norm, which contains all and only the norm 1 matrices.

Our Definition of Expressivity Let us define a general expressivity function of a set of matrices S as follows:

$$E(S) : S \subseteq \mathbb{R}^{N\times N} \to [0,1] \subset \mathbb{R}$$

where, when $E(S) = 1$, the maximum expressivity is reached, meaning that the set S is as expressive as $\mathbb{R}^{N\times N}$, and $E(S) = 0$ means S is not expressive at all, for example $S = \{0\}$. Furthermore an expressivity function E should have the following property: $s \subseteq S \Rightarrow E(s) \leq E(S)$

Let Ω be the surface of the L_2 unit sphere in $\mathbb{R}^{N\times N}$, Π a generic probability distribution over Ω and $W \sim \Pi$:

$$E(S) = 1 - \mathbb{E}_W[\|W - P_S(W)\|]$$

has all the above mentioned properties, and is indeed a well defined Expressivity function as we prove in **Theorem 1** That expected value can be expanded into:

$$\mathbb{E}_W[\|W - P_S(W)\|] = \int_\Omega \|W - P_S(W)\| P(W) dW$$

nevertheless in the most of the cases we don't need to compute $E(S)$ using the integral, since thanks to the central limit theorem: let $Z_i = \|W_i - P_S(W_i)\| \quad i = 1, ..., N$ be a succession of i.i.d random variables, where $W_i \sim \Pi$.

$$\frac{1}{M}\sum_{i=1}^{M} \|W_i - P_S(W_i)\| \approx \mathbb{E}_W[\|W - P_S(W)\|] \quad M \gg 1$$

The CLT can be used in practical since we can approximate $E(S)$ by simply applying

$$\frac{1}{M}\sum_{i=1}^{M} \Big\| \frac{U_i}{\|U_i\|} - P_S\Big(\frac{U_i}{\|U_i\|}\Big) \Big\|$$

and sampling U_i from a distribution $\tilde{\Pi}$ such that $W_i = \frac{U_i}{\|U_i\|} \sim \Pi$.

We indeed come up with two main choices for the distribution of W_i: the Gaussian distribution and the Uniform distribution, which both share a central role in the deep learning field.

Table 6. Expressivity of Kronecker Products $E(S)$

N	$U_i \sim \mathcal{U}(0,1)$	$U_i \sim \mathcal{N}(0,1)$
16	0.753	0.107
64	0.863	0.033
128	0.902	0.017
1024	0.964	0.002

Table 7. Expressivity of Monarch Matrices

N	$U_i \sim \mathcal{U}(0,1)$	$U_i \sim \mathcal{N}(0,1)$
16	0.784	0.367
64	0.871	0.208
256	0.93	0.11
1024	0.964	0.05

These results confirm what we expected, and what already existing theories predict, that Monarch matrices are actually more expressive than rank one.

To find these values we ran a simple PyTorch script, that sample U_i from $\tilde{\Pi}$, then computes σ_1, σ_2: the first and second singular values (such that $\sigma_1 \geq \sigma_2 \geq \sigma_3 ...$), and take the mean of all ratios: σ_2/σ_1 We can clearly note the difference between Uniform and Gaussian distribution, and we report a mathematical explanation for these results after the **Theorem 1** proof

```
# Python Pseudocode example to
    approximate E(S)
M = 100_000
p = 1024
q = 1024
res = []
for i in tqdm(range(M)):
    W = randn/rand (p, q)
    s1, s2 = sigma1_and_sigma2(W)
    ratio = s2 / s1
    res.append(ratio)

print(1 - mean(res)) # This is exactly
    E(S)
```

Theorem 1. *$E(S)$ is a well defined Expressivity Function meaning:*
1. $E(S) \leq 1$
2. $E(S) \geq 0$
3. $s \subseteq S \Rightarrow E(s) \leq E(S)$

Proof. 1. comes naturally from the positivity of the norm operator, in fact: $E(S) = 1 - \mathbb{E}[\|W - P_S(W)\|] \leq 1$
2. from the definition of the projection P_S:

$$\begin{aligned}
&\|W - P_S(W)\| \leq \|W\| = 1 \\
&\Rightarrow \quad \mathbb{E}_W[\|W - P_S(W)\|] \leq 1 \\
&\Rightarrow \quad -\mathbb{E}_W[\|W - P_S(W)\|] \geq -1 \\
&\Rightarrow \quad 1 - \mathbb{E}_W[\|W - P_S(W)\|] \geq 0 \\
&\Rightarrow \quad E(S) \geq 0
\end{aligned}$$

3. from the definition of projection:

$$\begin{aligned}
\|W - P_s(W)\| &= \underset{A \in s}{argmin} \|W - A\| \\
\|W - P_S(W)\| &= \underset{A \in S}{argmin} \|W - A\|
\end{aligned}$$

and using this property of min:

$$\begin{aligned}
&\underset{A \in S}{min} \|W - A\| \leq \underset{A \in s}{min} \|W - A\| \quad since \;\; s \subseteq S \\
&\Rightarrow \|W - P_S(W)\| \leq \|W - P_s(W)\| \\
&\Rightarrow \mathbb{E}_W(\|W - P_S(W)\|) \leq \mathbb{E}_W(\|W - P_s(W))\| \\
&\Rightarrow -\mathbb{E}_W(\|W - P_S(W)\|) \geq -\mathbb{E}_W(\|W - P_s(W)\|) \\
&\Rightarrow 1 - \mathbb{E}_W(\|W - P_S(W)\|) \geq 1 - \mathbb{E}_W(\|W - P_s(W)\|) \\
&\Rightarrow \quad E(S) \geq E(s)
\end{aligned}$$

□